

MuzzChat

Android chat technical exercise implementing a local two-user conversation with persistent messages.

Kotlin · Jetpack Compose · Room · Koin

What's Implemented

- Chat screen with local message persistence
- Sent and received message alignment
- Bubble grouping for consecutive messages from the same user within 20 seconds
- Date section headers when the gap between messages exceeds 1 hour
- Two-way messaging via manual user toggle
- Seed data designed to exercise grouping and sectioning rules

How to Run

- Open the project in Android Studio
- Sync Gradle
- Run the app module on an emulator or device with minSdk 29+

Assumptions

- The app models a conversation between two fixed local users
- Messages are stored only on-device
- Message ordering is based on local timestamps
- The “other user” is simulated via manual sender switching

Architecture

Single-screen Compose app with unidirectional state flow.

- **Presentation:** Compose UI + ViewModel
- **Domain:** models + BuildChatItemsUseCase
- **Data:** repository + Room

Given the scope of the exercise (one screen, two fixed users), I kept the project as a single module. Splitting it into feature or data modules would add structure, but not much value for this size of app.

Key Decisions

BuildChatItemsUseCase is isolated and JVM-testable

The chat list transformation (`List<Message> -> List<ChatItem>`) is kept outside the UI layer, with no Android dependencies. This keeps the grouping and sectioning rules in one place and makes them easy to test with plain unit tests.

Because the rules look both backward (`DateHeader`) and forward (`isGroupedWithNext`), I handled the transformation in a single indexed pass.

No separate effect layer

The screen only needs durable UI state (`items`, `inputText`, `currentUser`), so I did not introduce a separate one-off effects layer for this exercise.

Optimistic input clear

The input is cleared before the insert runs so sending feels immediate. In a production app, I would restore the input and surface the error if the write failed.

Testing

The core logic is covered by JVM unit tests:

- `BuildChatItemsUseCase`
- `MessageRepository`
- `ChatViewModel`

With More Time

- **Pagination.** Replace `observeAll()` with a paging-backed query to keep memory usage flat at scale.
- **Error handling on writes.** Wrap inserts in `try/catch`, emit an error state, and restore the input text if needed.
- **Compose UI tests.** Add instrumented tests for date headers and bubble grouping.
- **Per-bubble timestamps.** Show them conditionally on the last message of each group.
- **Splash screen guard.** Hold the splash explicitly if the seed grows larger.
- **Proper identity flow.** Replace manual sender switching with real user identity in a production scenario.